Bowyer–Watson Delaunay Triangulation

Delaunay Triangulation

Given a set of points

$$P = \{p_1, p_2, \ldots, p_n\},$$

the **Delaunay triangulation** connects the points into triangles such that:

Delaunay Property

The circumcircle of every triangle contains no other input point in its interior.

Equivalently, for every triangle T,

$$\text{Interior}(\text{Circumcircle}(T)) \cap P = \emptyset.$$

- Maximizes the minimum angle of the triangles.
- Widely used in computational geometry.
- Useful in mesh generation, graphics, FEM, and geographic modeling.

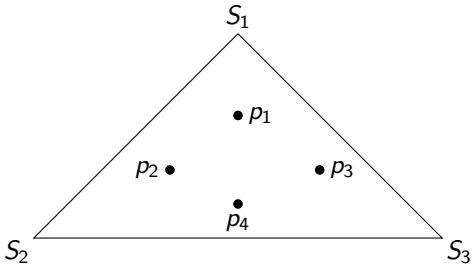
Bowyer–Watson Algorithm

The **Bowyer–Watson algorithm** constructs a Delaunay triangulation incrementally.

1. Start with a large **super-triangle**.
2. Insert input points one at a time.
3. Find triangles whose circumcircles contain the new point.
4. Remove these “bad” triangles.
5. The removed triangles form a polygonal cavity.
6. Connect the new point to the boundary of the cavity.
7. Remove triangles connected to the super-triangle.

Find bad triangles $\rightarrow$ Remove $\rightarrow$ Retriangulate

High-Level View



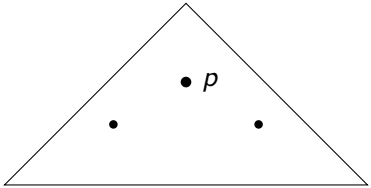
The super-triangle contains all input points.

Why a super-triangle?

It provides an initial triangulation and avoids special handling when the first points are inserted near the convex hull.

Step 1: Insert a New Point

Suppose a new point p is inserted.



For every existing triangle T, test:

$$p \in \text{Circumcircle}(T)?$$

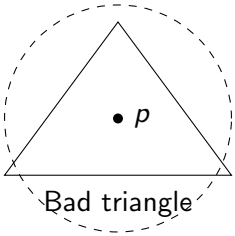
If yes, then T is a **bad triangle**.

Step 2: Find Bad Triangles

For a triangle T with circumcenter c and radius r:

$$\|p - c\|^2 < r^2$$

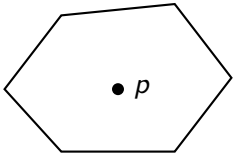
means that p lies inside the circumcircle.



The triangle must be removed because it violates the Delaunay property after inserting p.

Step 3: The Cavity

Suppose several triangles have circumcircles containing p.
These triangles are removed.



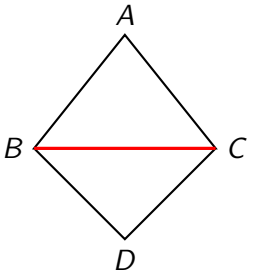
Polygonal cavity

The union of the bad triangles forms a cavity.

Only the **boundary edges** of the cavity are needed for retriangulation.

Step 4: Find the Cavity Boundary

Consider two bad triangles sharing an edge.



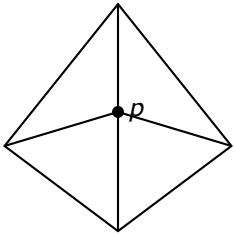
The edge BC occurs in **two bad triangles**.
Therefore it is an **internal edge** of the cavity and is discarded.

Boundary rule

An edge appearing only once among the bad triangles belongs to the cavity boundary.

Step 5: Retriangulate the Cavity

After removing the bad triangles, connect the new point p to every boundary edge.



Each boundary edge (u, v) generates a new triangle:

$$(u, v, p).$$

Complete Bowyer–Watson Algorithm

Algorithm 1 Bowyer–Watson

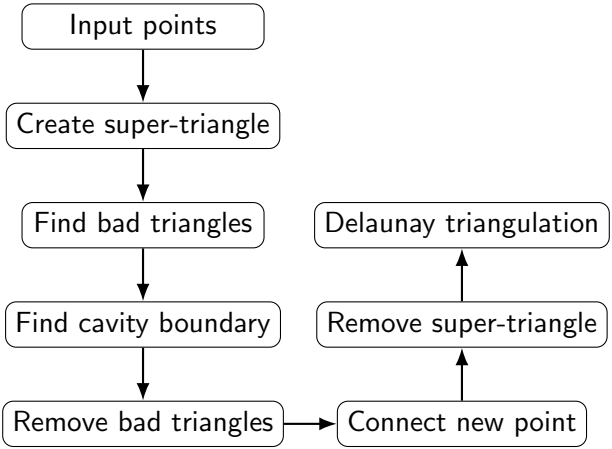
1: Create a super-triangle ST
2: $T \leftarrow \{ST\}$
3: **for all** points p **do**
4: $\quad Bad \leftarrow \emptyset$
5: $\quad$ **for all** triangles $t \in T$ **do**
6: $\quad\quad$ **if** p lies inside circumcircle of t **then**
7: $\quad\quad\quad Bad \leftarrow Bad \cup \{t\}$
8: $\quad\quad$ **end if**
9: $\quad$ **end for**
10: **end for**
11: $Boundary \leftarrow \emptyset$

Complete Bowyer–Watson Algorithm

Algorithm 2 Bowyer–Watson

```
1: for all t ∈ Bad do
2:     for all edges e of t do
3:         if e occurs only once in Bad then
4:             add e to Boundary
5:         end if
6:     end for
7: end for
8: Remove all triangles in Bad
9: for all edges (u, v) ∈ Boundary do
10:     Add triangle (u, v, p)
11: end for
12: Remove triangles connected to ST
13: return T
```

Bowyer–Watson: Complete Flow

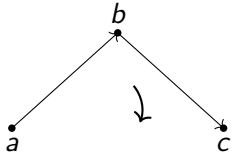


Geometric Predicate: Orientation

The orientation of three points is:

$$\text{orient}(a, b, c) = (b_x - a_x)(c_y - a_y) - (b_y - a_y)(c_x - a_x).$$

- orient > 0: counter-clockwise
- orient < 0: clockwise
- orient $= 0$: collinear



Orientation is useful for robust geometric computations.

Geometric Predicate: InCircle

The second important predicate is:

$$\text{inCircle}(a, b, c, p).$$

It determines whether p lies inside the circumcircle of triangle (a, b, c).
For a counter-clockwise triangle:

$$\det \begin{pmatrix} a_x & a_y & a_x^2 + a_y^2 & 1 \\ b_x & b_y & b_x^2 + b_y^2 & 1 \\ c_x & c_y & c_x^2 + c_y^2 & 1 \\ p_x & p_y & p_x^2 + p_y^2 & 1 \end{pmatrix} > 0$$

indicates that p is inside the circumcircle.

Implementation note

Robust Delaunay implementations often use exact or adaptive-precision geometric predicates rather than simple floating-point calculations.

Data Structures

A simple implementation can represent points, edges, and triangles as:

```
struct Point {
    double x, y;
};

struct Edge {
    Point *a;
    Point *b;
};

struct Triangle {
    Point *a;
    Point *b;
    Point *c;
};
```

Important operations include:

- inCircle()

Core Implementation Logic

```
for (Point *p : points) {

    vector<Triangle*> bad;

    // Find triangles whose circumcircle contains p
    for (Triangle *t : triangles) {
        if (inCircle(*p, *t))
            bad.push_back(t);
    }

    vector<Edge> boundary;

    // Find cavity boundary
    for (Triangle *t : bad) {
        for (Edge e : edges(t)) {

            if (countEdge(e, bad) == 1)
                boundary.push_back(e);
        }
    }

    // Remove bad triangles
```

Complexity

For a straightforward implementation:

$$\boxed{O(n^2)}$$

is a typical worst-case complexity.
Why?

- There are n point insertions.
- Each insertion may inspect many existing triangles.
- Finding the cavity can therefore take $O(n)$ work.

With appropriate spatial data structures and point-location methods, the practical performance can be significantly improved.

Key observation

The triangulation is updated locally around the newly inserted point.

Why Bowyer–Watson is Attractive

Advantages

- Conceptually simple
- Incremental
- Local cavity updates
- Easy to implement
- Natural geometric interpretation
- Useful for mesh generation

Challenges

- Point insertion is sequential
- Robust predicates are important
- Naive implementation can be $O(n^2)$
- Cavity searches can be expensive
- Parallel insertion requires conflict handling

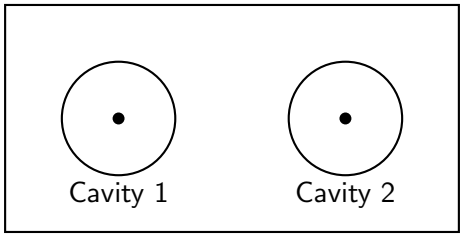
Parallelization Challenge

The basic algorithm is inherently incremental:

$$p_1 \rightarrow p_2 \rightarrow p_3 \rightarrow \cdots \rightarrow p_n$$

Each insertion modifies the triangulation.

However, multiple points may sometimes have **independent cavities**.



If the cavities do not overlap, their updates may be performed concurrently.

Parallel Bowyer–Watson: Basic Idea

A possible parallel approach is:

1. Select a batch of points.
2. Compute their cavities in parallel.
3. Detect overlapping/conflicting cavities.
4. Process independent cavities concurrently.
5. Merge the resulting triangulations.

$$\boxed{\text{Points} \rightarrow \text{Parallel cavity detection} \rightarrow \text{Conflict detection} \rightarrow \text{Parallel updates}}$$

This is an important direction for GPU and multicore implementations.

Applications

Delaunay triangulation is used in:

- Computational geometry
- Finite-element mesh generation
- Geographic information systems
- Terrain modeling
- Computer graphics
- Collision and proximity algorithms
- Voronoi diagram construction
- Scientific visualization

The Delaunay triangulation and Voronoi diagram are geometric duals.

Delaunay triangulation	$\longleftrightarrow$	Voronoi diagram

Summary

1. Start with a large super-triangle.
2. Insert points incrementally.
3. Find triangles whose circumcircles contain the new point.
4. Remove these bad triangles.
5. Extract the boundary of the resulting cavity.
6. Connect the new point to every boundary edge.
7. Remove triangles connected to the super-triangle.

Central idea

A new point invalidates a local set of triangles. Remove those triangles and retriangulate the resulting cavity.

$$\boxed{\text{Bowyer–Watson} = \text{Local cavity removal} + \text{retriangulation}}$$

Possible Next Step

Parallel Delaunay Triangulation

Interesting directions:

- Multicore Bowyer–Watson
- GPU-based Delaunay triangulation
- CUDA cavity detection
- Parallel conflict detection
- Divide-and-conquer Delaunay triangulation
- Incremental Delaunay on GPU
- FPGA implementation